

libmr — Relazione di progetto

Laboratorio 2 A, a.a. 2025-26

Fabio Tognarelli, 693132

Indice

1. Architettura generale	1
2. Interfaccia pubblica	2
2.1 Gestione degli Errori	3
3. Processi	4
4. Thread C11	5
5. Code interne	6
6. Protocollo sulle pipe	6
7. Raggruppamento per token	7
8. Formato e ordine dell'output	8
9. File di log	8
10. Test	8

1. Architettura generale

In questa sezione iniziale viene presentato il progetto, compreso della sua architettura e dei processi principali.

Partiamo con il dire che si tratta di un framework utilizzabile tramite la libreria statica `libmr(libmr.a)`, con header pubblico `include/mr.h`.

Il programma utente dovrà fornire in input solo due funzioni di callback, relative al processo di mapping e di reducing (`mr_mapper_t`, `mr_reducer_t`); una volta fatto cio' il framework leggerà l'input, eseguirà la pipeline di processi, raggrupperà per token e scriverà l'output risultante.

Vi sono **3 processi** principali, i quali comunicano via **pipe**

- **main** (processo chiamante): legge i file di input, serializza le righe verso il mapper, raccoglie i record dal reducer, li ordina e li scrive su un file di output;
- **mapper**: deserializza le righe, invoca la callback mapper su thread C11, serializza le coppie $\langle \text{token}, \text{valore} \rangle$ verso il reducer;
- **reducer**: accumula tutte le coppie, raggruppa per token, invoca la callback reducer una volta per token distinto, serializza i risultati verso il main.

Input:

- *file regolare* singolo,

- *directory* di sistema contenente piu' file di testo (non e' prevista esplorazione ricorsiva delle sottocartelle, si considerano quindi solo i file direttamente figli della cartella stessa), in **ordine lessicografico** sul nome.

Ogni file viene trattato come una sequenza di **righe logiche**: il carattere di fine riga: '\n' non fa parte del contenuto passato al mapper, ultima riga inclusa. File vuoti e righe vuote sono ammessi e gestiti correttamente. Per quanto riguarda la serializzazione della riga sulla pipe nel dettaglio consultare la sezione 6.

I puntatori in `mr_file_line_t` e in `mr_value_t` valgono solo durante la callback: emit copia token e byte opachi prima di ritornare.

Il seguente grafico mostra la **pipeline end-to-end** dei processi nel programma. La pipeline è lineare: il main è sia sorgente delle righe sia il collettore dei risultati. Sotto ogni stadio vi è l'ordine in cui si chiude il lato *write* (EOF sul *read* da main -> mapper). Senza i punti (1)–(3) un lettore resta bloccato in attesa di ricevere qualcosa in input. Gli worker paralleli gestiti dai thread non chiudono la pipe verso il reducer o verso il main.

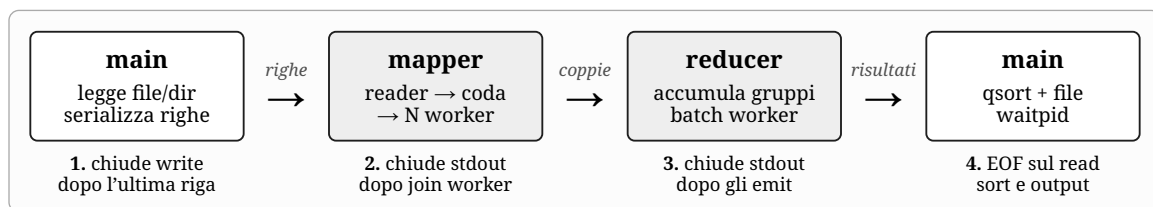


Figure 1: Pipeline di processi e chiusura delle pipe.

2. Interfaccia pubblica

Contratto pubblico di `mr.h` utilizzabile lato utente:

- `mr_t`: tipo opaco di una variabile `mr`
- `mr_attr_t`: contiene i valori degli attributi di `mr`. Si tratta di un valore copiabile; dopo `mr_create` il chiamante può eliminarlo o modificarlo senza effetto sull'istanza attuale.
- `mr_attr_init`: inizializza `attr` con valori di default
- `attr_setters`: lista di funzioni per settare ogni valore di `attr`
- `mr_create`: crea un `mr`
- `mr_start`: esecuzione di un `mr` (bloccante)
- `mr_destroy`: elimina un `mr`
- Valori Default di `mr_attr_init`: `mapper_threads` = 1, `reducer_threads` = 1, `queue_size` = 64, `log_file` = NULL (file `mr.log`). I setter rifiutano 0 per thread e coda.
- `mr_create` copia configurazione, puntatori alle callback e `user_arg`. `user_arg` non è interpretato; dopo `fork` le modifiche nei figli non sono visibili al padre né all'altro figlio.

- Valori intermedi e risultati: byte opachi (`data + size`). `size == 0` ammette `data == NULL`. Il framework non usa `strlen/strcmp/printf("%s")` su di essi. Il token è l'unica stringa C interpretata (alfanumerica ASCII, `'\0'` finale; il `'\0'` non sta sulla pipe).
- Ritorno: 0 successo, -1 errore, `errno` ove appropriato.

2.1 Gestione degli Errori

Verso il programma utente il contratto è semplice: ogni funzione pubblica restituisce 0 in caso di successo e -1 in caso di errore, con `errno` impostato. Chi chiama `mr_start` può quindi limitarsi a un `perror` e, se serve il dettaglio, leggere il file di log. Dietro questo contratto la gestione non può essere uniforme. Un errore può nascere nel processo principale, nel mapper o nel reducer; `errno` non attraversa il fork; una `close` o una scrittura sul log sovrascrivono `errno` della failure originale; un descrittore lasciato aperto nel processo sbagliato impedisce l'EOF e blocca la pipeline.

Per questo il framework distingue **chi può solo fallire** e **chi può parlare**.

Le funzioni di trasferimento e della coda (`io.c`, `queue.c`) sono silenziose, ovvero impostano `errno` e ritornano -1, senza scrivere sul log e senza `perror`. Un header con lunghezza negativa, nulla (per il token) o oltre i tetti della sezione 6 vale `EINVAL`; una `malloc` fallita vale `ENOMEM`; una `read` o `write` fallita lascia l'`errno` del kernel. L'EOF sulla pipe non è un errore: si segnala con `EOF_REACHED` e `errno` resta intatto. Queste funzioni girano sia nel main sia nei figli. Se loggassero, la stessa failure apparirebbe due volte, e i test sul protocollo (`tests/io.c`) non potrebbero verificare `EINVAL` in isolamento. Mapper, reducer e la lettura dell'input conoscono il contesto, quindi loggano. Nei processi figli un flag `ctx.error` ricorda che qualcosa è andato storto. I **thread worker non abortiscono da soli la pipeline**: segnano l'errore, finiscono il lavoro già in coda, poi il processo chiude `stdout` — così il lettore a valle riceve EOF — e ritorna -1. Se fallisce `mr_send_input`, il lato write verso il mapper **non** viene chiuso in `input.c`: quella chiusura spetta al parent. Un `close` in due posti diversi rischia un doppio `close` oppure un EOF prematuro, con il mapper che crede finito l'input mentre il main è ancora in errore.

Subito dopo il fork un fallimento di `dup2` o `close` non può tornare a `mr_start` come una funzione C normale: il figlio è già un processo distinto. In quel caso si usa `mr_child_fail`: una riga sul log e `_exit(1)`. Lo status 1 è l'unico canale con cui il figlio comunica il fallimento al padre.

Nel parent, una volta create le pipe (e, dopo i fork, i figli), lo stato ancora vivo sta in `mr_start_state_t`: descrittori aperti, record accumulati, PID, se i figli esistono. Ogni path di errore salva `errno` **prima** di loggare o chiudere, poi chiama `mr_start_cleanup`: si tratta appunto di una funzione di cleanup, la quale chiude ciò che è ancora aperto, libera i record e, se i figli sono già stati creati, li attende con `waitpid`. Prima dei fork il cleanup resta manuale: non ha senso fare `waitpid` su processi che non esistono.

Un figlio uscito con codice diverso da 0, o ucciso da un segnale, diventa `errno = EIO` per il chiamante. Il parent non rilogga la stessa failure se il figlio è già uscito con 1 — l’ha già scritta sul log. Logga invece i segnali (WIFSIGNALED), perché in quel caso il figlio può non essere riuscito a scrivere.

Un messaggio troncato o una lunghezza invalida, **dentro** `io.c`, è `EINVAL`. Se accade nel mapper o nel reducer, il figlio termina con `_exit(1)` e `mr_start` restituisce `EIO`. Lo stesso `EINVAL` visto dal programma utente vale per argomenti sbagliati, path di input non valido, o un secondo `mr_start` sullo stesso handle: cioè per errori che non hanno avviato, o non devono riavviare, la pipeline. Una pipe o un fork falliti **nel main** invece arrivano al chiamante con l’`errno` del kernel.

Errore	Quando	Effetto
EINVAL	Puntatori NULL; thread o <code>queue_size</code> pari a 0; path di input non valido; secondo <code>mr_start</code> sullo stesso handle. In <code>io.c</code> : lunghezze negative o oltre i tetti.	Ritorno -1, <code>errno = EINVAL</code> . Se l’errore è nel figlio, il chiamante di <code>mr_start</code> vede <code>EIO</code> .
ENOMEM	Fallimento di <code>malloc</code> , <code>realloc</code> o <code>calloc</code> .	Ritorno -1 con <code>errno</code> del sistema. Cleanup di quanto già allocato.
syscall main	Fallimento di <code>pipe</code> , <code>fork</code> , <code>close</code> , <code>open</code> , <code>waitpid</code> nel processo principale.	Ritorno -1 con l’ <code>errno</code> del kernel. Log e <code>mr_start_cleanup</code> (chiusura FD, <code>waitpid</code> sui figli già creati).
EIO	Figlio uscito con codice diverso da 0 (<code>mr_child_fail</code> o <code>ctx.error</code>) oppure terminato da un segnale.	Il figlio ha già loggato (salvo i segnali). Il main in <code>waitpid</code> imposta <code>errno = EIO</code> .

Scelta. `io.c` e `queue.c` non loggano. Il parent usa uno snapshot (`mr_start_state_t`) e un cleanup unico; i figli in setup usano `mr_child_fail`. Si salva `errno` prima di ogni `log/close/waitpid`.

Perché. Lo stesso I/O vive in tre processi: un log in foglia produce righe doppie. Un cleanup sparso lascia descrittori aperti e la pipeline si blocca in attesa di EOF. Dopo una `close`, `errno` non è più quello della failure. `_exit(1)` è l’unico canale figlio-padre; il parent lo traduce in `EIO` e non ripete il messaggio già scritto dal figlio.

Scelta. Dopo un errore in `mr_start`, l’handle `mr_t` non è riutilizzabile: occorre `mr_destroy` e un nuovo `mr_create` prima di un altro `mr_start`.

Perché. `mr_start` crea processi, pipe e stato interno (record accumulati, FD). Un secondo avvio sullo stesso handle mescolerebbe descrittori e PID. Il contratto one-shot evita un reset parziale difficile da rendere corretto.

3. Processi

Qui vengono descritti i processi e tutte le chiamate di sistema che vengono effettuate su di essi. Una volta che viene lanciato `mr_start`:

1. Creazione di tre pipe: `main_to_mapper`, `mapper_to_reducer`, `reducer_to_main`;

2. fork del mapper; nel figlio dup2 su stdin/stdout, chiusura di **tutti** i FD delle tre pipe (restano stdin/stdout già ridiretti);
3. fork del reducer; stesso schema (stdin ← read di mapper_to_reducer, stdout → write di reducer_to_main);
4. nel main restano aperti solo il write verso il mapper e il read dal reducer; gli altri sei estremi si chiudono;
5. invio righe, chiusura del write verso il mapper, raccolta risultati fino a EOF, qsort, scrittura file, waitpid di mapper e reducer.

I figli eseguono mapper_process_main / reducer_process_main e poi _exit. Non c'è nessuna exec. I fork avvengono **prima** di qualsiasi thrd_create (nel main non ci sono thread C11 del framework, sono presenti solo in mapper e reducer).

Se il fork del reducer fallisce, il main chiude le pipe e fa waitpid del mapper già avviato. I figli in errore di dup2/close usano _exit dopo aver scritto sul log.

4. Thread C11

Header <threads.h>. Niente pthread_*. Firma dei thread: int (*)(void *).

Mapper. Un thread *reader* legge da stdin i messaggi riga, li inserisce nella coda (sezione 5) e alla EOF chiama mr_queue_close. N = mapper_threads worker fanno pop, ricostruiscono mr_file_line_t locale, invocano la callback del mapper, emettono coppie. La scrittura su stdout è gestita da una mtx_t: un messaggio logico non deve mescolarsi con un altro e viene quindi sincronizzato. In emit viene fatta la validazione del token (non vuoto, solo esadecimale). Chiusura stdout: dopo thrd_join di reader e worker (passo 2 in figura 1). Id di log: reader = 0, worker = 1...N.

Reducer. Allo stesso modo del mapper, un thread *reader* legge coppie da stdin e le inserisce nei gruppi (sezione 7). I worker **non** partono in parallelo al reader: si fa thrd_join del reader (EOF = nessuna coppia ulteriore), poi si invoca la callback del reader su ogni gruppo per token.

Dopo il join del reader, i gruppi si elaborano a **batch** di al più reducer_threads: per ogni batch si creano i worker (un gruppo a testa), poi si fa join e si passa al batch successivo.

Questo perché. La specifica del progetto impone il reduce solo a gruppi completi: non c'è produzione concorrente di gruppi durante il reduce, quindi non serve una coda produttore-consumatore in questa fase. Il batch usa il parametro reducer_threads senza tenere thread idle durante tutta la lettura.

Anche qui viene usata una mutex su stdout: più worker dello stesso batch possono fare emit insieme ed è un'eventualità che va evitata.

5. Code interne

Si tratta di una coda interna ai processi, utilizzata solo nel **processo mapper**, per coordinare reader e worker. Non è una pipe e non è condivisa fra processi, in quanto come già detto rimane interna. Capacità = queue_size (numero di elementi, non byte). Buffer circolare di puntatori a righe allocate in heap.

Se la coda è piena, il reader attende not_full; se invece è vuota, il worker attende not_empty; close sblocca i consumer e i pop successivi falliscono (fine lavoro).

```
typedef struct {  
    void **items;  
    size_t cap, head, tail, count;  
    mtx_t mtx;  
    cnd_t not_full, not_empty;  
    int closed;  
} mr_queue_t;
```

Come tipo di coda è stata scelta una **bounded queue**: in questo modo il push si blocca a coda piena; queue_size è il numero massimo di righe in volo nel mapper.

Il tetto esiste perché i worker possono essere più lenti del reader: senza un limite, ogni riga già letta da stdin resterebbe in heap in attesa, e su un file grande la memoria del mapper crescerebbe senza controllo. Con queue_size il push si ferma a coda piena, il reader smette di svuotare la pipe e il main a monte rallenta: restano in volo al più tante righe quante l'utente ha chiesto, e il parametro ha un effetto concreto sulla memoria e sul ritmo della pipeline.

Il reducer non usa questa coda: l'accumulo è la tabella dei gruppi (sezione 7).

6. Protocollo sulle pipe

Tre tipi di messaggio, tutti length-prefixed. readn / writen ripetono read/write fino a n byte (gestione EINTR e trasferimenti parziali). Fine flusso = chiusura pipe, non un messaggio sentinella. Lunghezze in header: int; si rifiutano valori negativi e valori oltre i tetti sotto, **prima** della conversione a size_t. token_len == 0 è invalido. Il token sulla pipe è senza '\0'; il ricevente alloca token_len+1 e lo aggiunge. Payload opachi: esattamente value_len byte, nienti impliciti.

Scelta. Tetti: MR_MAX_TOKEN_LEN = 1 MiB, MR_MAX_VALUE_LEN = 64 MiB, MR_MAX_LINE_LEN = 64 MiB, MR_MAX_NAME_LEN = 4096.

Perché. La spec chiede limiti ragionevoli documentati. 4 KiB copre path Linux tipici; 1 MiB sul token è già oltre ogni chiave alfanumerica attesa; 64 MiB su riga e valore opaco ammette record binari grandi senza malloc da int non validato.

Messaggio **riga** (main → mapper):

```
typedef struct {  
    int file_name_len;  
    int line_len;
```

```

    unsigned long line_number;
} mr_line_header_t;
/* poi: file_name_len byte di nome, line_len byte di riga (senza '\n') */
Messaggio coppia (mapper → reducer) e risultato (reducer → main): stesso header;
il risultato riusa il campo valore per i byte opachi emessi dal reducer.
typedef struct {
    int token_len;
    int value_len;
} mr_pair_header_t;
/* poi: token_len byte di token, value_len byte opachi */
value_len == 0: nessun payload, puntatore NULL lato ricevente.

```

7. Raggruppamento per token

Nel reader del reducer, ogni coppia entra in `add_or_create_group`. Un gruppo = un token distinto; la callback reducer parte **una volta** per gruppo, con tutti i valori, solo dopo EOF.

```

typedef struct {
    char *token;
    void **data;      /* copia heap di ogni processed_token */
    size_t *sizes;
    size_t count;
    size_t cap;
} token_group_t;

```

Lookup: scansione lineare con `strcmp` sul token (stringa C già terminata in ricezione). Se esiste, si accoda una **copia** dei byte; se `value_size == 0`, si accoda NULL. Se non esiste, `realloc` dell'array di gruppi e nuovo `token_group_t` (cap iniziale 4, raddoppio). Dopo EOF: `qsort` dei gruppi per token (`strcmp`), poi i batch della sezione 4. L'ordine sulla pipe verso il main **non** è il contratto di determinismo (i worker di uno stesso batch emettono in concorrenza): l'ordine del file è nella sezione 8.

Scelta. Tabella = array dinamico di `token_group_t` con lookup lineare; ordinamento dei gruppi con `qsort` dopo EOF.

Perché. Il reduce parte solo a input esaurito: non serve una struttura concorrente durante l'inserimento. Il token è una stringa C, quindi `strcmp` è lecito (non sui valori). Le copie heap rispettano il contratto di `emit` (il chiamante può riusare i buffer). L'array è sufficiente per il numero di chiavi distinte atteso su un corpus da laboratorio; l'ordinamento dei gruppi rende deterministica l'assegnazione ai batch.

8. Formato e ordine dell'output

Il file di output è binario, stesso layout del messaggio risultato (sezione 6): `int` lunghezza token, `byte` del token (senza `'\0'`), `int` lunghezza risultato, `byte` opachi del risultato. Non è testo; `cat` non è significativo.

Il `main` accumula i record in un array (`record_from_reducer_t`), poi scrive il file.

Scelta. I record nel file sono in ordine lessicografico sul token (`strcmp`). Se il reducer fa più `emit` per lo stesso token, l'ordine relativo nel file è l'ordine di quelle chiamate.

Perché. La spec chiede output deterministico a parità di input, callback e attributi, e di definire l'ordine se ci sono più risultati per chiave. Il sort nel `main` è indipendente dallo scheduling dei worker reducer: l'ordine di arrivo sulla pipe non è usato come ordine di file. Più `emit` sullo stesso token restano nell'ordine in cui la callback li ha prodotti (stesso worker, sequenza naturale).

9. File di log

Default: `mr.log` nella directory di lavoro. Apertura `O_APPEND`. Formato riga:

`[YYYY-MM-DD HH:MM:SS] [processo] [thread_id] [evento] messaggio`

- processo: `main`, `mapper`, `reducer`;
- thread_id: 0 per `main` e per i reader; 1..N per i worker;
- evento (principali): `pipe`, `fork`, `thrd_start`, `thrd_end`, `close`, `output`, `stats`, `error`;
- stats: righe inviate al mapper, coppie emesse, token distinti, record finali.

Esempio: `[2026-06-21 14:30:01] [main] [0] [pipe] created 3 pipes`

Eventi minimi della spec coperti: creazione pipe e processi, avvio/fine thread, apertura/chiusura file di input e output, i quattro contatori, errori.

Scelta. Mutua esclusione inter-processo con semaforo POSIX named: `sem_open("/<pid>", ...)` nel `main` all'apertura del log; ogni `mr_log_write` fa `sem_wait` / `written` / `sem_post`. Il nome usa il PID del processo che crea il log, così istanze concorrenti di programmi diversi non condividono lo stesso semaforo.

Perché. Tre processi (e più thread nel mapper/reducer) scrivono lo stesso file: senza lock le righe si mescolano. Il semaforo named è ereditabile dopo `fork` (stesso mapping) e copre sia thread sia processi, come previsto dalla spec. `written` evita righe spezzate da `write` parziali.

10. Test

`make test` compila `libmr.a` se serve, costruisce i sei eseguibili in `tests/` e li lancia in sequenza. Al primo fallimento la batteria si ferma (exit diverso da 0). I test di integrazione usano mapper e reducer propri (conteggio, byte opachi, mapper che non emette): il framework non incorpora logica di word-count.

Livello	Eseguibile	Cosa verifica
I/O e log	tests/log	formato riga; default <code>mr.log</code> ; scritture concorrenti da processi distinti col semaforo
	tests/input	file singolo; directory (ordine lessicografico); file vuoto
	tests/io	readn/writen; messaggi riga, coppia, risultato; EOF; header/payload troncati; <code>mr_validate_len</code>
Stadi isolati	tests/mapper	processo mapper in un figlio: una riga in ingresso, coppia attesa su stdout
	tests/reducer	processo reducer: coppie già serializzate, raggruppamento, un emit per token
Integrazione	tests/integration	<code>mr_create/mr_start/mr_destroy</code> : word-count, directory, determinismo, valori opachi, mapper silenzioso, coda piccola con più thread, input inesistente e one-shot

Dettaglio dei comandi: README.